

Knowledge reasoning based on graph neural networks with multi-layer top-p message passing and sparse negative sampling[☆]

Wenjie Liu^{a,c,b},^{*} Zhijie Ren^a, Liang Chen^a

^a School of Software, Nanjing University of Information Science and Technology, Nanjing, 210044, China

^b Jiangsu Collaborative Innovation Center of Atmospheric Environment and Equipment Technology (CICAET), Nanjing University of Information Science and Technology, Nanjing, 210044, China

^c Jiangsu Province Engineering Research Center of Advanced Computing and Intelligent Services, Nanjing University of Information Science and Technology, Nanjing, 210044, China

ARTICLE INFO

Keywords:

Graph neural networks
Knowledge reasoning
Multi-layer top-p message passing
Sparse negative sampling

ABSTRACT

Graph Neural Networks (GNNs) have received increasing interest in knowledge reasoning since they can learn the structure and semantic information of graphs. However, as the number of hops increases, the number of entities grows exponentially, resulting in a sharp rise in resource costs. Furthermore, the presence of irrelevant entities during message passing can accumulate noise, thereby diminishing the model's accuracy. To solve these problems, a knowledge reasoning model based on graph neural networks with multi-layer top-p message passing and sparse negative sampling is proposed. To be specific, we designed a dynamic top-p message-passing strategy that dynamically samples key entities related to the query based on their probability distribution, thereby reducing high resource costs. Then the similarity-based negative sampling is applied to dense entities in the knowledge graph, while random sampling is used for sparse entities, which enhances the model's ability to identify irrelevant entities. Extensive experiments conducted on three datasets (WN18RR, FB15k-237, and NELL-995) demonstrate our model outperforms other SOTA methods in knowledge reasoning, achieving an average improvement of 5.16%, 6.16% and 6.71% in MRR, Hits@1 and Hits@10 respectively, and it also has a great advantage in training efficiency than SOTA GNN-based methods, with the average improvement of 11.99% in training time. Our model not only offers a novel method for knowledge reasoning, but also contributes valuable insights for the advancement of knowledge graph technologies.

1. Introduction

A knowledge graph is composed of a large number of triples formed by head entity, relation, and tail entity, representing knowledge and facts of the real world. Given a query, we hope that the knowledge reasoning model can fully exploit the hidden relation in the knowledge graph and infer the answer entity through reasoning strategies. In the context of artificial intelligence development, knowledge graph plays an important role in fields such as intelligent question answering [1], recommendation systems [2], and drug discovery [3]. These applications highlight the critical need for effective knowledge reasoning to bridge the gap between raw graph data and actionable insights. Consequently, knowledge reasoning has become a research focus in the field of knowledge graphs.

Knowledge reasoning is currently mainly divided into three methods based on reasoning evidence: triple-based, path-based and graph neural networks-based. In triple-based methods, there are methods based on translational distance, tensor decomposition, and neural networks. Triple-based methods leverage individual triples as reasoning units, offering efficient yet localized reasoning processes. The translation distance-based methods, use the mapping vectors of head entity, tail entity and relation to represent semantic information, such as TransE [4], TransR [5], TorusE [6], RotatE [7], etc. In tensor decomposition-based models, the entire knowledge graph is represented as a three-dimensional adjacency matrix consisting of smaller unitary third-order tensors, where each tensor corresponds to a triple, where each tensor corresponds to a triple. The tensor elements are assigned a value of 1 if the factual knowledge exists; otherwise, they are set to 0. Models of this

[☆] This work is supported by the National Natural Science Foundation of China (62071240), the Innovation Program for Quantum Science and Technology (2021ZD0302901), the Natural Science Foundation of Jiangsu Province (BK20231142), and the Priority Academic Program Development of Jiangsu Higher Education Institutions (PAPD).

^{*} Corresponding author.

E-mail addresses: wenjieliu@nuist.edu.cn (W. Liu), 202212490288@nuist.edu.cn (Z. Ren), 20211249390@nuist.edu.cn (L. Chen).

<https://doi.org/10.1016/j.knosys.2025.113063>

Received 1 August 2024; Received in revised form 14 December 2024; Accepted 20 January 2025

Available online 27 January 2025

0950-7051/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

type include RESCAL [8], DistMult [9], ComplEx [10], TuckER [11], etc. Neural network-based methods such as ConvE [12], ConvKB [13], ConvR [14], etc., typically transform knowledge reasoning tasks into standard deep learning tasks. By adjusting the weight parameters of the neural network according to the input triple information, these methods can identify the relevant patterns among the triples. Compared to triple-based and tensor decomposition-based knowledge reasoning methods, neural network-based methods are more expressive in knowledge reasoning, but on the other hand, they also bring problems such as high time complexity and poor interpretability. The methods above are trained on individual triple information, which ignores the structure of the graph and the semantic information of other entities, resulting in a lack of interpretability in reasoning. The path-based methods provide local evidence for reasoning [15] such as PTransE [16], Path-RNN [17], DeepPath [18], etc. These methods provide reasoning with a certain degree of interpretability. Although effective, these methods are limited to sequential reasoning and cannot capture subgraph-level structural patterns, reducing their flexibility in more complex scenarios.

Compared to relational paths, graph structures encompass richer semantic information [19]. Graph Neural Network is a type of neural network model specifically tailored for the representation learning of graph-structured data [20]. It can achieve reasoning tasks by learning the structure and semantic information of the graph through message passing [21]. To more accurately model triple data in knowledge graph, researchers have introduced graph neural networks into knowledge reasoning, such as R-GCN [22], CompGCN [23], KBAT [24], etc. However, these models leverage only the first-order or second-order neighbor information of the central entity is used for reasoning. Wang et al. [25] believe that each entity contains rich semantic information, and it is necessary to integrate various semantics in the knowledge graph in order to learn more comprehensive node embeddings. As the order increases, the number of entities grows exponentially, significantly escalating the computational complexity of graph neural network models. During the message-passing process, the performance of the model is impacted due to the aggregation of representations of a large number of entities irrelevant to reasoning. Moreover, in the negative sampling phase, a strategy of randomly replacing head or tail entities [26] is adopted in most methods, which may lead to a large difference between negative and positive samples. This can result in a slow convergence rate for the model and make it difficult for the model to discriminate against more challenging negative samples. Another critical challenge lies in the negative sampling phase. Current strategies, such as randomly replacing head or tail entities, may lead to uninformative negative samples, slowing convergence and reducing the model's ability to handle challenging reasoning scenarios.

To address the problems above, a knowledge reasoning model based on multi-layer top-p message passing and sparse negative sampling is proposed. First, at each layer, the probability distribution of entities with respect to the query is calculated and dynamically optimized the probability threshold p . Then, the core entities from the probability distribution based on the probability threshold p are dynamically sampled, and the entities at the tail of the distribution are ignored, which reduces the number of entities. Next, a graph neural network is used for message passing on the sampled subgraphs. In the negative sampling phase, a density-based negative sampling method is adopted to improve the quality of negative samples. For entities with higher density, some negative samples with high similarity to the sample are sampled, which helps to improve the model's ability to distinguish between positive and negative samples. Considering the sparsity of knowledge graphs, for entities with lower density, which have lower similarity with surrounding entities, random negative sampling is adopted.

The main contributions of this paper can be summarized as follows:

- A multi-layer top-p message passing method, which ensures that entities can propagate further while retaining entities more relevant to the query, and reduces the number of entities irrelevant to the reasoning process, is proposed to improve inferential performance while reducing model training time.

- To enhance the model's ability to distinguish between positive and negative samples, a sparsity-based negative sampling scheme is proposed. For entities with high density, negative samples with high similarity to the positive sample are selected, while random negative sampling is applied to entities with low density.
- To verify the performance of the model, multiple experiments were conducted on datasets WN18RR, FB15k-237, and NELL-995, and the results are analyzed from the metrics of MRR, Hits@1, and Hits@10. The experimental results show that the performance of our model is superior to the state-of-the-art models in most aspects. Additionally, the training efficiency is higher than the latest GNN-based reasoning models.

Here is the layout of this article. In Section 2, we introduce the related work of knowledge reasoning. Section 3 elaborates on our proposed model. Our experimental setup and results are detailedly presented in Section 4. Finally, Section 5 of this paper presents the conclusion and outlines the directions of future research.

2. Related work

2.1. Triple-based methods

Triple-based reasoning methods rely on the patterns and properties of individual triple in the knowledge graph for reasoning. Translational distance model RotatE [7] projects triples into the complex vector space and defines relation embeddings as rotational vectors, making the mapping operation of triples equivalent to the rotation of entity embeddings along the coordinate axis. Tensor decomposition models view the knowledge graph as a three-dimensional adjacency matrix. RESCAL [8] represents relation vectors with full-rank matrices, which requires extensive computation. DistMult [9] replaces the full-rank matrix in RESCAL with a diagonal matrix for relation embedding, reducing the parameter space for representation learning and making the model easier to train. Neural network-based models, represented by convolutional neural networks, commonly reformulate the link prediction task into a standard deep learning task. ConvE [12] uses a neural network to learn shared parameters based on the input triple information, identifying the associative patterns between triples. InteractE [27] employs three stacking methods for the input head entity vector and relation vector: stacking, cyclic permutation, and element-wise cross arrangement. Additionally, it enhances the interaction between entity and relationship vectors by replacing regular convolutions with cyclic convolutions. However, the methods above train based on individual triple, lacking interpretability in reasoning.

2.2. Path-based methods

Relational paths are collections of triples, which, compared to triple-based methods, can provide local evidence for reasoning to enhance interpretability. PTransE [16] improves the TransE model by modeling multi-hop relational paths to achieve reasoning tasks. MINERVA [28] uses reinforcement learning to search for answer paths based on input questions for reasoning, without precomputed paths. PathCon [29] retrieves all paths between target entities and models path information through message passing, but its cost is expensive. Unlike path information including entity information, rule information only models relational paths without including entity information in the paths. Such methods can generalize to reasoning for entirely new entities. DRUM [30] learns chained rules between entities by learning confidence scores for the corresponding rules to achieve inference. However, paths and chained rules only model sequential information and cannot learn complex patterns such as subgraph structures to serve reasoning.

Path-based methods leverage relational paths to enhance interpretability by providing local evidence for reasoning. However, these methods often face high resource costs and limitations in modeling complex patterns like subgraph structures, restricting their ability to fully utilize the rich semantics in knowledge graphs.

2.3. GNN-based methods

In light of the outstanding performance of graph neural networks in learning graph-structured data, numerous knowledge graph reasoning methods based on graph neural networks have been proposed. For knowledge graphs, which are a type of heterogeneous graph, R-GCN [22], based on the convolutional neural network GCN, introduces corresponding weight matrices for each relation in the knowledge graph. During the aggregation of information, neighbor information is aggregated with weights according to the respective weight matrices. However, as the variety of relations increases, the number of relation transformation matrices also increases, leading to an explosion in the number of parameters in the model. CompGCN [23] designs a set of base vectors and represents all relation embeddings in the knowledge graph through a weighted combination of these base vectors, thereby significantly reducing the parameter complexity caused by a large number of relations. R-GCN [22] takes into account different relationship information but overlooks the direction of relations. VR-GCN [31] considers both the type and direction of relationships and applies this concept to R-GCN, enabling entities to distinguish whether neighboring entities are head entities or tail entities when aggregating neighbor embeddings. Since the importance of aggregated neighbor entity information varies for the central entity, various models based on the Graph Attention Network (GAT) [32] have been proposed. KBGAT [33] takes into account the relational attention beyond multiple hops of the target entity, propagating attention iteratively in a hierarchical manner. This helps the model to more comprehensively understand the relationships in the knowledge graph and improve the performance of the model.

The models above only rely on the one-hop or two-hop neighbor information of entities for reasoning, but the knowledge graph contains rich structural and semantic information that can serve knowledge reasoning. Given that the paths between entities contain rich reasoning information, NBFNet [34] models the path information between entities as the representation of the target entity, defining the representation of node pairs as the generalized sum of all path representations. The model has achieved good performance in both inductive and transitive reasoning, yet it requires path searching between entities for each query, leading to high resource costs. RED-GNN [35] dynamically constructs a directional subgraph starting from the central query entity and propagates layer by layer through graph neural networks. In this subgraph, the information between the query entity and multiple answer entities is modeled all at once. The final layer's entity representation represents the subgraph, containing the inference information required for the query. However, as the number of layers increases, the number of entities grows exponentially, leading to significant resource consumption. NA-KGR [36] addresses the issue of excessive noise introduced in inference tasks by stacking multiple layers of graph neural networks on a graph. It utilizes the weighted nature of graph attention to encode the neighboring entities most likely to impact inference through a two-stage graph attention mechanism positively. While this reduces the influence of entities irrelevant to inference, these entities remain part of the message-passing computation. As the number of layers in the model increases, the resource cost becomes significant. One of the challenges currently faced by graph neural network-based knowledge representation learning models is the high resource cost for large-scale graphs. The problem addressed by the method proposed in this paper is how to reduce the resource cost while ensuring good model performance.

3. Method

3.1. Task and notation definitions

In a knowledge graph, a triple object is represented as (h, r, t) , where h, r, t represents the head entity, relation and tail entity respectively.

Table 1
Definitions of notations.

Notations	Definitions
Q	The set of query triples
(e_q, r_q, e_a)	A query triple in Q with query entity e_q , query relation r_q and answer entity e_a
L	The set of sampling layers
l_i	Sampling layer l_i
p	The probability threshold p for top-p sampling
T^{l_i}	The representation of query triple in sampling layer l_i
$p(e^{l_i})$	The entity probability distribution in sampling layer l_i
$\rho(e^{l_i})$	The relevance of neighboring entities with the query triple in sampling layer l_i
$h_e^{l_i}$	The representation of entity e in sampling layer l_i
d_e	The neighbor connection density of entity
S	The sparsity of entity
$p(e_i)$	The probability of entity e_i

Definition 1 (Knowledge Reasoning). In a KG (h, r, t) , find the answer entity referring to a query: $(e_q, r_q, ?) \rightarrow (e_q, r_q, e_a)$.

For example, given a query (*Inception*, *Directed by*, ?), the reasoning process can be described as follows: “*Inception* $\xrightarrow{\text{Has_Actor}}$ Dom Cobb $\xrightarrow{\text{Role_creator}}$ Christopher Nolan”, identifying “Christopher Nolan” as the answer entity.

Graph Neural Networks (GNNs) can be represented by the graph structure $G = (V, E)$, where V and E are denoted as the sets of nodes and edges respectively.

Definition 2 (Message Passing). Let a node $V_i \in V$ and all its neighboring nodes $V_j \in V$ with edges $E_{ji} \in E$, pass the message from node V_j to node V_i : $V_j \xrightarrow{\text{message}_{ji}} V_i$.

The most frequently used notations in this paper are shown in Table 1.

3.2. Model overview

To incorporate more useful high-order entity information into the central entity during the GNN learning process and improve the training efficiency, a knowledge reasoning model based on Graph neural networks with multi-layer top-p message passing and sparse negative sampling is proposed. First, in each layer, a dynamically optimized probability threshold p is determined, and the probability distribution of the entities relative to the query is computed. Then, the key entities are dynamically sampled based on the probability threshold p , and the entities at the tail of the probability distribution are ignored, which reduces the number of entities involved in message passing. Next, a graph neural network is used for message passing on the sampled subgraphs. During the negative sampling phase, a density-based negative sampling method is adopted to improve the quality of negative samples. For high-density entities, negative samples with greater similarity to the positive samples are selected, enhancing the model's ability to distinguish between positive and negative samples. Random negative sampling is adopted for the entities with lower density because these entities have lower similarity with surrounding entities.

The overall framework of the model is shown in Fig. 1. Here it is assumed that the sampling layer is 2. At the beginning, the representations of entities are initialized by the TransH model. In the next step, multi-layer top-p sampling is performed on each query entity, followed by a round of graph neural network message passing after sampling of each layer, and the representations of entities are updated. In the negative sampling phase, a sparsity-based negative sampling algorithm is performed to generate negative samples, both positive and negative samples are used for training the model. The loss function is calculated based on the model's scoring of these samples, and backpropagation is conducted to update the parameters of our model.

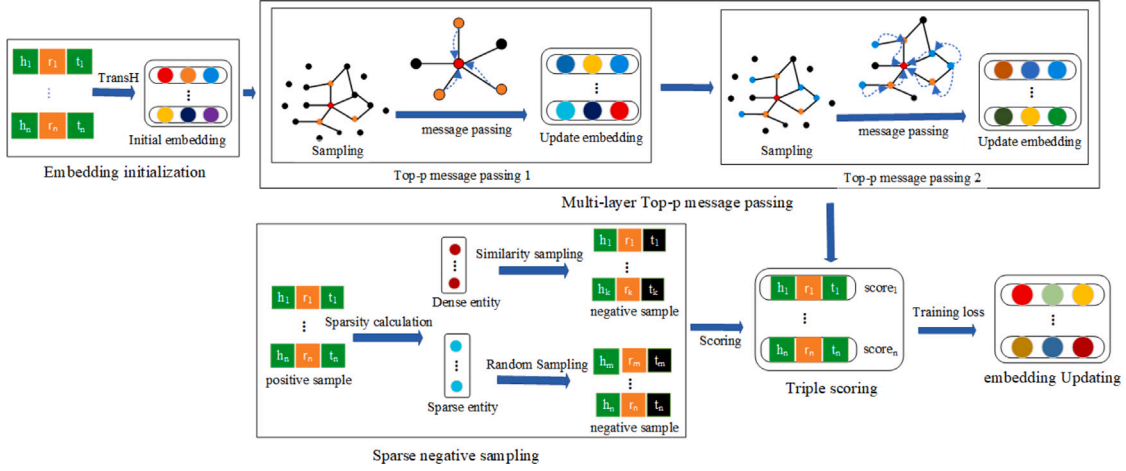


Fig. 1. The overall framework of the model.

3.3. Multi-layer top-p message passing

To reduce resource resources while ensuring performance, the key is to determine the number of entities to sample at each layer. We propose sampling the entities most relevant to the query for message passing to obtain local evidence related to the query. In a fixed-number top-k sampling algorithm, it is easy to include samples with low relevance to the query or exclude some important ones. To address this issue, we propose a multi-layer top-p sampling method that selects candidates by dynamically optimizing the threshold p , meaning the number of selected elements can dynamically change according to the distribution characteristics, rather than being a fixed quantity. This approach provides greater flexibility to adapt to the actual data distribution, particularly in knowledge graphs, which often display uneven distributions and long-tail effects. By truncating the unreliable tail of the probability distribution of entities relative to the query, highly relevant entities are retained. Additionally, the top-p algorithm, by focusing on the probability distribution rather than a fixed number of top scores, helps mitigate overfitting to specific features in the training data.

Specifically, the sampling layers $L = \{l_0, l_1, \dots, l_n\}$ are set for top-p sampling, where layer l_0 only contains the query entity. For each subsequent layer l_i , the candidate entities C^l are derived from the neighbors of the sampled entities S^{l-1} in the previous layer, as defined below:

$$C^l = \bigcup_{e \in S^{l-1}} \text{neighbors}(e) \setminus \bigcup_{k=0}^{l-1} S^k \quad (1)$$

For each candidate entity $e' \in C^l$, its relevance to the query triple is calculated. The query triple representation T_i^l in layer l_i is

$$T_i^l = h_e^{l-1} + r - t \quad (2)$$

where h_e^{l-1} is the embedding of the candidate entity in the $(l-1)$ th layer, r is the embedding of the query relation (fixed across all layers), and t is the embedding of the fixed target entity defined in the initial query triple (h, r, t) . Then, we calculate the relevance $\rho(e^l)$ of these neighboring entities with the query triple T_i^l , as follows:

$$\rho(e^l) = h_e^{l-1} \cdot T_j^{l_i} \quad (3)$$

The relevance of all neighboring entities is normalized to obtain the entity probability distribution $p(e^l)$,

$$p(e_i^l) = \frac{\rho(e_i^l)}{\sum_{j \in C^l} \rho(e_j^l)} \quad (4)$$

Next, the neighboring entities are sorted based on their probabilities. The cumulative calculation of the sum of probabilities begins with the entity that has the largest probability. The sampling in layer l_i does not stop until the total probability of the sampled neighbor entities exceeds p . Unlike the method of running a multi-layer GNN on the graph after sampling in all layers, we utilize a graph neural network to perform message passing on the subgraph formed by the neighbor entities sampled in current layer and the entities sampled in previous layers, updating the representations of the entities. This approach reduces the resource cost of message passing. The graph attention mechanism is introduced when aggregating the information of neighboring entities, with the formulas as follows:

$$s = \text{relu}(w_1^l [h_{e_s}^{l-1}, h_{e_o}^{l-1}, h_{r_{so}}^{l-1}, h_{r}^{l-1}] + b_1^{l-1}) \quad (5)$$

$$a_{e_s e_o}^l = \sigma(w_2^l s + b_2^l) \quad (6)$$

where $a_{e_s e_o}^l$ represents the weight value of the source entity relative to its neighboring entities, relu , σ denote activation functions, w_1^l , w_2^l are learnable parameter matrices of layer l , f represents the concatenation operation, $h_{e_s}^{l-1}$ is the representation of the source entity at layer l_{i-1} , $h_{e_o}^{l-1}$ is the representation of the source entity's neighboring entities at layer l_{i-1} , $h_{r_{so}}^{l-1}$ is the representation of the relationship between the source entity and its neighboring entities, h_r^{l-1} is the representation of the target relationship, b_1^l , b_2^l are learnable biases of layer l_i . The representation $h_{e_s}^l$ of the entities at layer l_i is updated

$$a^l = \sum_{o \in N_s} w^l a_{e_s e_o}^l h_{e_o}^{l-1} \quad (7)$$

$$h_{e_s}^l = \text{AGG}(h_{e_s}^{l-1}, a^l) \quad (8)$$

where a^l represents the weighted aggregated representation of the neighboring entities' information, calculated as a scalar value for each neighbor. Specifically, the scalar weights $a_{e_s e_o}^l$ are combined with the neighboring entities' embeddings to form a^l , which ensures that the aggregation process retains the relational importance between source and neighboring entities, N_s represents all neighboring entities of the source entity, w^l is a trainable parameter matrix, and AGG represents the aggregation function (e.g., mean, max, or sum).

Unlike running a multi-layer GNN on the entire graph, this approach performs message passing on progressively reduced entity sets, exponentially lowering computational overhead while maintaining reasoning quality. The schematic diagram of multi-layer top-p message passing is shown in Fig. 2, and Algorithm 1 gives the details of the multi-layer top-p message passing algorithm.

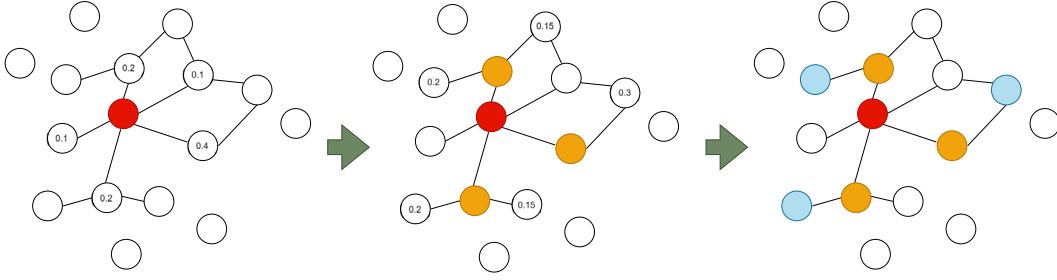


Fig. 2. Schematic diagram of multi-layer top-p sampling.

Algorithm 1 Multi-layer top-p message passing

Input: knowledge graph G , $query(e_q, r_q, ?)$, $S^0 = \{e_q\}$, sampling layers L , top-p sampling threshold p .

Output: the set of all representations $h_{e_s}^L$

- 1: **for** l in L **do**
- 2: get neighboring entities set: $S^l = S_{neb}^{l-1} \setminus S^{l-1}$, set of edge: $E^l = \{(e_h, r, e_t) | e_h, e_t \in (S^l \cup S^{l-1})\}$
- 3: logits computation: $\rho(e_i^l) = \{h_{e_i}^l \cdot (h_{e_q}^l + h_{r_q}^l - h_{e_a}^l), e_i \in S^l\}$,

$$p(e_i^l) = \frac{\rho(e_i^l)}{\sum_{j=1}^N \rho(e_j^l)}$$
- 4: sampling top-p entities: $S_{top_p}^l = \{top_p(S^l)\}$
- 5: get MESSAGING entities: $S_{MESS}^l = S^{l-1} \cup S_{top_p}^l$
- 6: MESSAGING:

$$h_{(e_s, r, e_o)}^l = \{MESS(h_{e_s}^{l-1}, h_r^{l-1}, h_{e_o}^{l-1}) | e_h, e_t \in S_{MESS}^l\}$$
- 7: AGGREGATE: $h_{e_s}^l = AGG(h_{e_s}^{l-1}, h_{(e_s, r, e_o)}^l)$
- 8: **end for**
- 9: **return** the set of all representations: $h_{e_s}^L$

3.4. Negative sampling method based on sparsity

In large-scale knowledge graphs, most entities are sparse, but there are also a small number of dense entities. For entities with higher density, their neighboring entities exhibit higher similarity. The multi-layer message passing and aggregation in GNNs often result in adjacent entities having increasingly similar representations, making it challenging for the model to accurately identify the target entity during reasoning tasks. To mitigate this, the model is trained to effectively distinguish between these similar samples. In the context of negative sampling, randomly replacing the head or tail entity for high-density entities often generates negative samples that significantly differ from positive samples, which can hinder both the model's performance and convergence. To address this limitation, we introduce a sparsity-based negative sampling method to enhance the quality of negative samples, as outlined in Algorithm 2. For high-density entities, we prioritize sampling negative samples that are similar to the positive ones, as these entities have numerous connections, and their neighboring entities are closely aligned in the feature space. After multi-layer GNN message passing, these similarities increase further, making it difficult for the model to distinguish these similar entities. Therefore, choosing negative samples that are similar to the positive samples forces the model to learn finer-grained features, thereby enhancing its discriminative ability. For low-density entities, due to their fewer neighbors and lower similarity in the feature space, an efficient random negative sampling method is adopted to provide sufficient diversity while simplifying the computation and reducing the cost.

In this paper, the formula for calculating the neighbor connection density is as follows:

$$d_e = L_e / N_e \quad (9)$$

where N_e represents the number of neighbors of the entity, and L_e represents the number of connections between entity neighbors. The formula for calculating the average entity neighbor connection density $\bar{D}_{KG}$ in the knowledge graph is as follows:

$$\bar{D}_{KG} = \frac{1}{N} \sum_{i=0}^N d_{ei} \quad (10)$$

To gain a deeper understanding of the structural properties of entities within the knowledge graph, we further calculate the sparsity-based on density

$$S = 1/d_e \quad (11)$$

we measure the sparsity of entities based on their neighbor connection density. If the sparsity is below the average, the entity is considered sparse; otherwise, it is classified as dense. For each dense query entity e_q in the sample, obtain all of its one-hop and two-hop neighbor entity sets N_{e_q} , and calculate the cosine similarity between these entities and the corresponding answer entity e_a of the query entity

$$\cos_similar(e_a, N_{e_q}) = [\cos_similar(e_a, n_{e_q}), n_{e_q} \in N_{e_q}] \quad (12)$$

the entity with the highest cosine similarity is used as the replacement entity s_{e_q} for the negative sample

$$s_{e_q} = \arg \max(\cos_similar(e_a, N_{e_q})) \quad (13)$$

By adopting adaptive negative sampling methods based on the sparsity of entities, we improve the quality of negative samples and enhance the training effectiveness and reasoning performance of the model.

3.5. Optimization of the threshold p

Setting a fixed threshold p for top- p message passing may overlook long-distance dependencies and the overall graph structure. Additionally, important local entities and diverse node characteristics may be ignored, leading to the loss of critical information. To address this, we propose a dynamic adjustment of p to select the most relevant entities for message passing while controlling information loss.

To ensure that the selected entities retain the most significant information, we define an information loss threshold δ . The cumulative probability of the selected top- p entities must satisfy the following condition:

$$\sum_{i \in \text{top-}p} p(e_i) \geq 1 - \delta \quad (14)$$

This constraint ensures that at least $1 - \delta$ of the total information content is preserved among the selected entities. To enforce this, we calculate the cumulative distribution function (CDF) for the sorted entities based on their relevance scores. The CDF is defined as:

$$\text{CDF}(e_i) = \sum_{j=1}^i p(e_j) \quad (15)$$

where $p(e_j)$ represents the relevance score of entity e_j . Based on the CDF, we determine the smallest p such that:

$$\text{CDF}(e_p) \geq 1 - \delta \quad (16)$$

This method dynamically adjusts the threshold p based on the distribution of relevance scores, ensuring that the selected entities retain sufficient information while reducing unnecessary computations.

Algorithm 2 Sparse Negative Sampling Method

Input: knowledge graph $KG = (V, E)$, sparsity threshold θ

Output: negative samples S_{neg}

```

1: Calculate sparsity:
2: for all entity  $e \in V$  do
3:    $d_e = \frac{L_e}{N_e}$ 
4:   classify as sparse if  $d_e < \frac{1}{N} \sum_{i=0}^N d_{ei}$ , else dense
5: end for
6: Generate negative samples:
7: for all entity  $e$  do
8:   if dense entity then
9:     Calculate cosine similarity  $n \in N_e$ 
10:    Select negative sample:
11:     $s_e = \arg \max(\cos\_similar(e, N_e))$ 
12:   else
13:     Randomly select a negative sample from  $V$ 
14:   end if
15:   Append  $s_e$  to  $S_{neg}$ 
16: end for
17: return the set of negative samples  $S_{neg}$ 

```

3.6. Model training

The Margin Ranking Loss function is used as the training loss function, aiming to ensure that the score of positive samples is higher than that of negative samples. The model's parameters are updated through stochastic gradient descent. The formula of loss function is as follows:

$$L = \sum_{i=1}^N \max(0, \text{score}(n_i) - \text{score}(p_i) + Y) \quad (17)$$

where $\text{score}(n_i)$ is the score of negative samples, $\text{score}(p_i)$ is the score of positive samples, and Y is the margin hyperparameter, it is used to define the expected minimum margin of difference in scores between positive and negative samples for the model.

Algorithm 3 Dynamic Optimization of Threshold p

Input: Probabilities $p(e_i)$ for each entity e_i , Information loss threshold δ .

Output: Optimal threshold p .

```

1: Sort entities by  $p(e_i)$  in descending order.
2: Compute the CDF:

```

$$\text{CDF}(e_i) = \sum_{j=1}^i p(e_j)$$

```

3: Find the smallest  $p$  such that:

```

$$\text{CDF}(e_p) \geq 1 - \delta$$

```

4: return the top- $p$  entities that satisfy the constraint.

```

4. Experiments

In this section, we validated the effectiveness of our model on several link prediction task datasets. The experimental hardware environment used in this paper is the Ubuntu 18.04 LTS, Intel (R) Xeon

Table 2

Dataset statistics.

Dataset	Entities	Relations	Train	Validation	Test	Total
WN18RR	40 943	11	21 705	3034	3134	27 873
FB15k-237	14 541	237	68 028	17 535	20 466	106 029
NELL-995	74 536	200	37 420	543	2818	40 781

(R) CPU E5-2658A v3 @ 2.20 GHz 2.20 GHz dual core CPU, NVIDIA A100 GPU, 64 GB memory. The software environment was built using the Pytorch 1.8.0 framework.

4.1. Datasets

We used three widely used datasets for link prediction tasks, WN18RR [12], FB15k237 [37], and NELL-995 [18]. The statistical information of the datasets is shown in Table 2.

- **WN18RR** [12] is a subset of WordNet, a comprehensive lexical database of English. WN18RR addresses the test set leakage issue present in the original WN18 dataset by removing inverse relations, resulting in a more challenging evaluation scenario. WN18RR comprises 93,003 triplets, encompassing 40,943 entities and 11 relations, thus providing a robust assessment for link prediction models due to its heightened difficulty and realistic settings.
- **FB15k237** [37] mitigates the problem of test set leakage by excluding redundant and inverse relations. The dataset contains 272,115 triplets with 14,541 entities and 237 relations. FB15k-237 offers a more demanding and realistic evaluation context, enabling a precise assessment of model performance in practical applications.
- **NELL-995** [18] is part of the Never-Ending Language Learning (NELL) project, which continuously learns to extract structured information from the web. NELL-995 dataset includes 154,213 triplets, with 75,492 entities and 200 relations. NELL-995 poses significant challenges due to its high relational diversity and the ongoing learning process it represents.

4.2. Baselines and evaluation metrics

We compare our model with 7 baselines of link prediction models. In non-GNN methods, we chose the triple-based model RotatE [7], tensor decomposition-based model DisMult [9], neural network-based model ConvE [12] and path-based model DRUM [30]. In GNN methods, we chose CompGCN [23], RED-GNN [35], and NA-KGR [36].

- **RotatE** [7] is one of the most effective models in distance-based models, partly due to the rotation operation introduced during modeling, which significantly enhances the model's expressive power.
- **DisMult** [9] is an important benchmark model for link prediction tasks, which uses diagonal matrices instead of full rank matrices in RESCAL to represent relation embeddings, reducing the parameter space for reasoning and making the model easier to train.
- **ConvE** [12] is the first model to use multi-layer convolutional neural networks to solve link prediction tasks,
- **DRUM** [30] learns chained rules based on paths between target entities by learning confidence scores for the corresponding rules to achieve reasoning.
- **CompGCN** [23] incorporates the relations in the graph into reasoning, using them as embedding vectors and node vectors for joint learning. In addition, it enriches the types of edges by adding reverse and self-recurrent relation types to the edges, further enhancing the model's expressive power.

Table 3
Experimental results on the benchmark dataset.

Model	WN18RR			FB15k237			NELL-995		
	MRR	Hits@1	Hits@10	MRR	Hits@1	Hits@10	MRR	Hits@1	Hits@10
RotatE	0.475	42.6	57.4	0.338	24.1	53.3	0.401	34.4	50.1
DisMult	0.465	43.2	53.2	0.313	22.4	49.0	0.281	19.9	44.6
ConvE	0.427	39.0	50.8	0.305	21.9	47.6	0.491	40.3	61.3
DRUM	0.486	42.5	58.6	0.343	25.5	51.6	–	–	–
CompGCN	0.479	44.3	54.6	0.355	26.4	53.5	–	–	–
RED-GNN	0.533	48.5	62.4	0.374	28.3	55.8	<u>0.543</u>	<u>47.6</u>	<u>65.1</u>
NA-KGR	0.486	46.2	60.2	0.532	46.9	64.0	0.462	34.9	56.2
Ours	0.559	49.8	67.6	<u>0.423</u>	<u>43.4</u>	<u>58.3</u>	0.559	49.6	66.2

- **RED-GNN** [35] performs reasoning based on subgraph information by dynamically constructing subgraphs between the central entity and multiple answer entities. It utilizes graph neural networks for message passing, and the representations of entities in the final layer serve as the subgraph representations for the corresponding query.
- **NA-KGR** [36] utilizes two stages of graph attention mechanisms, leveraging the weighted properties of graph attention to encode neighboring entities that are most likely to have a positive impact on inference.

In our method, the number of model sampling layers L in set $\{4, 5, 6, 7, 8\}$, the top- p sampling probability threshold p is dynamically optimized, the batch size is in set $\{32, 64, 128\}$ and the number of model iterations to 50. Selection of evaluation metrics: MRR , $Hits@1$ and $Hits@10$.

MRR represents the average of the reciprocal of the ranks of the actual answers in the prediction results for each link prediction task, with its specific definition as follows:

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{q} \quad (18)$$

the value of MRR ranges between 0 and 1, with a larger value indicating better predictive performance.

$Hits@K$ represents the ratio at which the ranking of the actual answer in the prediction is equal to or less than the threshold value K , with its specific definition as follows:

$$Hits@K = \frac{|\{q \in Q, q \leq K\}|}{|Q|} \quad (19)$$

the higher the value of $Hits@K$, the better the predictive performance of the model.

4.3. Results and analysis

Table 3 shows the results on the dataset and indicates that methods based on graph neural networks have significant performance improvements compared to methods based on translation distance, tensor decomposition, and neural networks. In graph neural network-based models, CompGCN requires loading the entire knowledge graph for calculation, due to the large number of entities in the NELL-995 dataset, it exceeds memory limits. For each pair of target entities, DRUM needs to search for paths between the targets to extract candidate rules. However, the presence of a large number of entities in the NELL-995 dataset causes memory exhaustion.

Our model achieved the experimental results on the WN18RR dataset with 8 sampling layers and an optimized sampling probability threshold of 0.8. On the NELL-995 dataset, the experimental results were obtained with 6 sampling layers and an optimized sampling probability threshold of 0.75. On the FB15K-237 dataset, the experimental results were achieved with 7 sampling layers and an optimized sampling probability threshold of 0.8. Our model's experimental results on the WN18RR and NELL-995 datasets are better than the baseline results. However, on the FB15K-237 dataset, due to the relatively small number of entities and the large number of relations, the relationships

Table 4
Comparison of training time (min) for different GNN-based models.

Model	WN18RR		FB15k237		NELL-995	
	Time	Hits@10	Time	Hits@10	Time	Hits@10
RED-GNN	<u>460</u>	<u>58.6</u>	<u>3697</u>	52.4	<u>1588</u>	<u>61.6</u>
NA-KGR	498	57.3	3867	56.3	1707	53.9
Ours	423	59.9	3369	<u>53.6</u>	1423	62.8

Table 5
Statistics on entity sparsity.

Dataset	$0 < S \leq 1$	$S > 1$	$S > \bar{S}$
WN18RR	23 431	9	385
FB15k-237	13 058	204	4217
NELL-995	30 014	325	1403

between entities are more complex, and the probability distribution differences between entities are small. Dynamic threshold sampling leads to the loss of key entities, making it difficult to obtain good training results. Overall, our model achieved average improvements of 5.16%, 6.16%, and 6.71% in MRR , $Hits@1$, and $Hits@10$ respectively on three datasets.

To demonstrate the advantages of our method in improving GNN-based model training efficiency, we set the number of model layers to 6 and the number of training iterations to 20.

The runtime and performance of each model are shown in Table 4. Although RED-GNN constructs subgraphs for multiple query triplets simultaneously, which improves training efficiency to some extent, it propagates over all entities, leading to significant resource consumption. NA-KGR, while achieving performance advantages on the FB15k-237 dataset, faces challenges on the NELL-995 dataset due to the vast number of entities. Performing message passing across all entities not only incurs high resource costs but also fails to yield good performance. Our method effectively reduces resource costs through a multi-layered top- p message-passing strategy and outperforms other baseline models in both performance and training time, achieving an 11.9% reduction in training time, demonstrating its efficiency in managing large-scale knowledge graphs.

4.4. Ablation study

To investigate the impact of negative sampling strategies based on sparsity on model performance, we conducted an ablation study and provided statistics on the sparsity of entities in three benchmark datasets, as shown in Table 5. The majority of entities in these benchmark datasets are sparse. We evaluated the performance of the model with 7 sampling layers using different negative sampling strategies. The results of different sample strategies are shown in Fig. 3(a) and (b), sparsity-based negative sampling strategies significantly improve performance compared to random negative sampling strategies, albeit with a slight increase in resource cost. Achieving further performance gains would require similarity-based negative sampling for all samples, which is computationally expensive and offers only limited improvements. Therefore, similarity-based negative sampling is applied only to

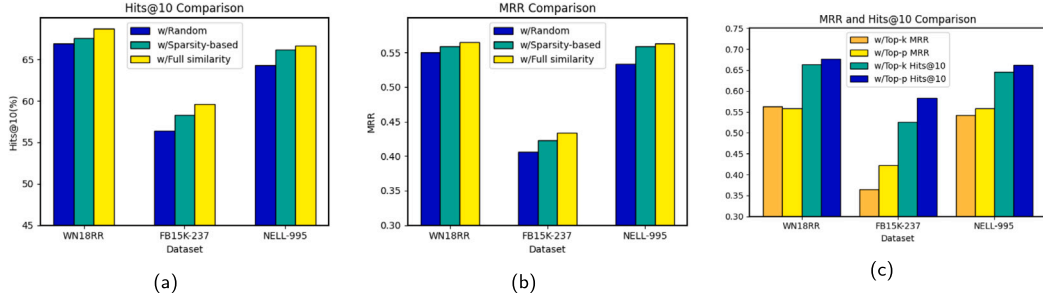


Fig. 3. Ablation study of different strategies. (a) Ablation of sampling strategies on Hits@10. (b) Ablation of sampling strategies on MRR. (c) Ablation of message passing strategies on Hits@10 and MRR.

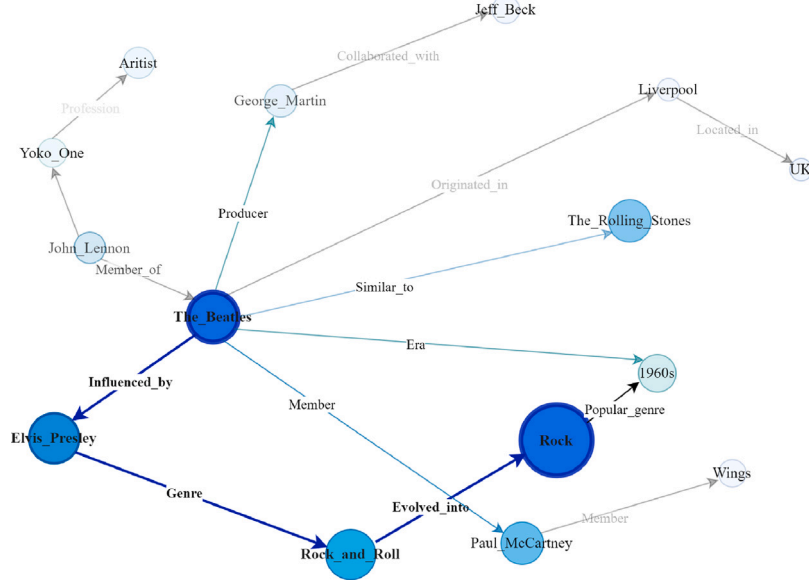


Fig. 4. The figure illustrates the inference process for the query (The Beatles, Genre, ?). The dark blue nodes represent the target nodes predicted by the model. The color intensity and size of the nodes indicate the node attention scores; the darker and larger the node, the higher the attention weight. Similarly, the color intensity and thickness of the edges represent the contribution scores; the darker and thicker the edge, the higher the contribution. The figure shows how the final answer, “Rock”, is derived through multiple relationships and intermediate nodes. Each arrow points from the tail entity to the head entity, with the predicate on the edge indicating the relationship between them, forming a triplet.

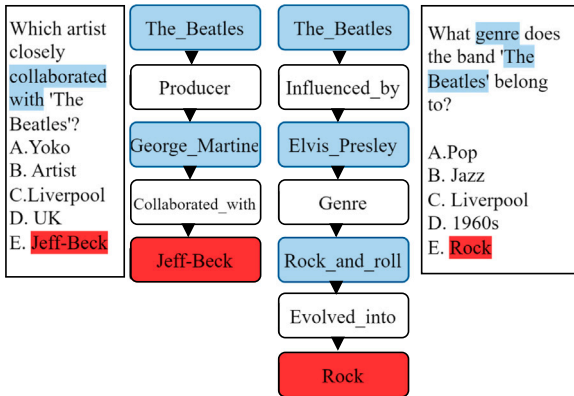


Fig. 5. Case study on model interpretability.

dense entities, while an efficient random negative sampling strategy is employed for other entities.

The ablation study highlights the advantages and trade-offs of different negative sampling strategies. The Sparse Negative Sampling Strategy significantly improves performance compared to random sampling,

with a moderate increase in resource cost. Although full similarity-based negative sampling yields the best results, it comes with high resource cost and offers only modest improvement over sparsity-based sampling. Consequently, our proposed Sparse Negative Sampling Strategy, which employs similarity-based sampling for dense entities and random sampling for sparse entities, strikes a balance between performance and efficiency. The results of the ablation study demonstrate that this strategy not only significantly enhances the model’s reasoning and generalization capabilities but also maintains a reasonable resource cost, thereby validating the rationality and effectiveness of our approach.

Moreover, to investigate the impact of different message-passing strategies on model performance, we conducted an ablation study on top-p message passing and compared the combination of sparse sampling with top-k message passing.

The performance comparison of different message-passing strategies across three benchmark datasets is demonstrated in Fig. 3(c). The results indicate that the combination of sparse sampling and top-p message passing outperforms in the majority of cases. This suggests that top-p message passing is more effective in preserving crucial information and enhancing model performance while maintaining reasonable resource costs within a reasonable range.

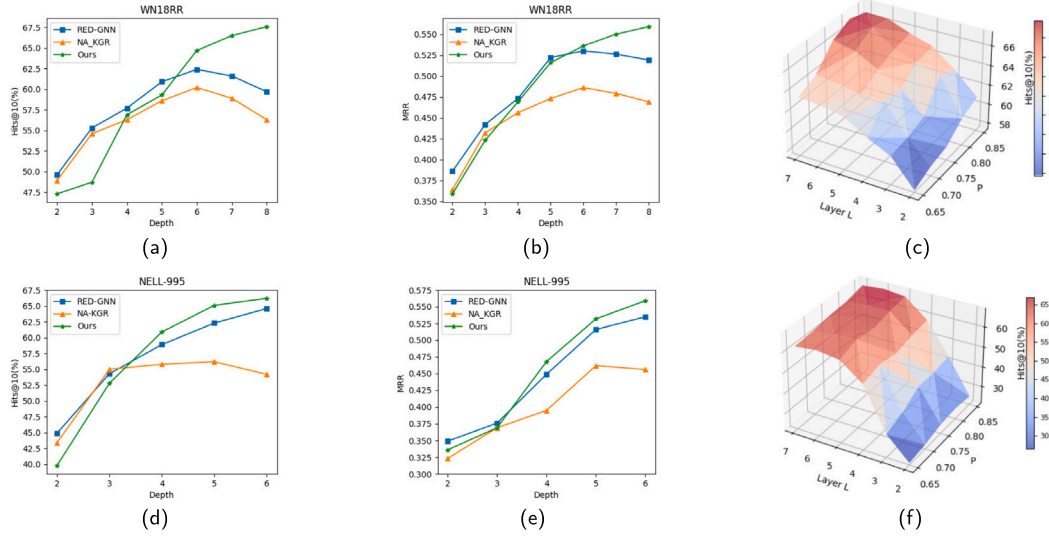


Fig. 6. Performance with respect to different parameters. (a) Effect of L on Hits@10 for WN18RR (b) Effect of L on MRR for WN18RR (c) Effect of L and p on Hits@10 for WN18RR (d) Effect of L on Hits@10 for NELL-995 (e) Effect of L on MRR for NELL-995 (f) Effect of L and p on Hits@10 for NELL-995.

4.5. Model interpretability

Model interpretability is crucial for understanding and validating the predictions [38]. The inference graph we extracted provides a graphical explanation of the model's predictions. As illustrated in Subsection C, each entity in the inference graph is assigned an attention weight. We selected the query 'What genre does the band *The Beatles* belong to?' from the FB15k-237 dataset for analysis.

The *The Beatles* node represents the query subject. The *Influenced_by* edge connects to *Elvis Presley*, indicating that *The Beatles* were influenced by *Elvis Presley*. The *Elvis Presley* node connects to *Rock and Roll* through the *Genre* edge, indicating that *Elvis Presley's* genre is *Rock and Roll*. The *Rock and Roll* node connects to *Rock* through the *Evolved into* edge, indicating that *Rock and Roll* evolved into *Rock*.

To illustrate the inference process more clearly, we provide the following visualizations: The intricate relationships between *The Beatles* and other related entities are illustrated in Fig. 4. Through multi-layered nodes and edges, we can clearly observe how the model synthesizes multiple pieces of information to infer that *The Beatles' genre is Rock*. The color intensity of each node represents its attention score, with darker colors indicating higher importance in the inference process. Similarly, the color intensity of each edge represents its contribution score, with darker colors indicating higher significance in the reasoning. This visualization ensures the transparency and interpretability of the entire inference process, enabling a detailed understanding of the model's reasoning path and decision-making basis in detail. Additionally, we exemplified the aforementioned question, demonstrating that our model correctly answers the question and provides a reasonable path as evidence, as shown in Fig. 5. In the left example, the model connects the question entities and the answer entity in a sequential chain to facilitate reasoning. In contrast, the right example demonstrates how the model utilizes previously unmentioned entities to bridge the gap between the question entity and the answer entities.

4.6. Parameter sensitivity

To further investigate the effect of hyperparameters on model performance, we vary the input sampling layers L and utilize the dynamic optimization strategy for determining the threshold p . Experiments are conducted on three datasets to evaluate the impact of the input sampling layers L and the threshold p on model performance.

- **Input sampling layers L :** In GNN-based models, to demonstrate the importance of higher-order entity information for reasoning, the impact of depth of message passing on the performance is investigated, as shown in Fig. 6(a), (b), (d) and (e). We set L to 2,3,4,5,6,7,8 and compared the performance of GNN-based methods under different depths of message passing. In FB15K-237 and NELL-995 datasets, as the order of entities increases, the performance of the model shows an upward trend, proving the importance of aggregating high-order entity information for inference. When the number of layers is less than 6, the performance gradually improves. However, once the layer count exceeds 6, performance begins to decline. This is due to the number of entities involved, leading to an increase in irrelevant information for inference. Our method maintains robust performance even when the depth exceeds 6 layers, demonstrating its effectiveness in filtering out irrelevant entities and enhancing reasoning capabilities.
- **Threshold p :** To investigate the impact of the dynamic sampling threshold p on model performance, we conducted experiments with different layers, as shown in Fig. 6(c) and (f). We used a dynamic adjustment algorithm to determine the appropriate p values during the training process. The results indicate that at the same depth, a larger threshold p can result in better performance because it allows more relevant entities to be considered. However, as the value of p increases, the number of entities included grows significantly, leading to an increase in irrelevant entities, which hampers the model's reasoning ability and limits its performance. Furthermore, as both the depth L and the sampling threshold p increase, there is an exponential growth in the number of entities considered, placing considerable strain on the computational efficiency and resource demands. Therefore, a balance must be struck in choosing appropriate values for the layers L and the dynamic threshold p . The 3D plots in (c) and (f) illustrate these trends, showing that while performance can be enhanced by the increase of L and p , there is a diminishing return and potential performance drop when these parameters are too high.

5. Conclusion

In this paper, a knowledge reasoning model based on graph neural networks with multi-layer top- p message passing and sparsity-based

negative sampling is proposed. The model employs a multi-layer top-p message-passing strategy to ensure that entity information can propagate further while reducing the number of entities irrelevant to reasoning during the graph neural network message-passing process. Through the sparsity-based negative sampling strategy, the quality of the generated negative samples is improved, enhancing the model's ability to distinguish between challenging positive and negative samples. Our model not only offers a novel method for knowledge reasoning but also contributes valuable insights for the advancement of knowledge graph technologies.

Our model demonstrates significant advantages in the field of knowledge reasoning, enhancing both accuracy and efficiency. Due to the sparsity of rare entity processing and the complexity of multimodal data, knowledge reasoning remains an exceedingly challenging task. There is still much work to be explored in this area. Below, we outline some potential research directions.

- **Integrating External Knowledge.** Integrating external knowledge from modalities such as text, images, or temporal data can significantly enhance the reasoning capabilities of knowledge graph models. Multimodal data provides richer contextual information, enabling the model to better understand entities and relationships. For example, textual descriptions can add semantic depth, and visual features can offer complementary insights. This integration enriches knowledge representation and improves generalization across diverse domains and tasks.
- **Real-Time Updates in Dynamic Graphs.** Another area for further exploration is the application of the model to dynamic knowledge graphs, where entities and relations evolve over time. Incorporating real-time updates into the reasoning process would require the development of more efficient methods for incremental learning and continuous adaptation to new information.
- **Optimizing for Low-Resource Environments.** Optimizing for low-resource environments is crucial for deploying knowledge reasoning models on edge devices or in scenarios with limited computational resources. Techniques like model pruning can be applied to reduce redundant parameters and streamline computations in GNN layers, significantly lowering memory and processing demands. Quantization further minimizes resource usage by converting weights and activations to lower-precision formats, while knowledge distillation allows a smaller, more efficient model to learn from a larger, high-performing one. Combined with efficient graph sampling or pooling methods, these strategies ensure that the model remains effective and resource-friendly, making it suitable for real-world applications under constrained conditions.

CRedit authorship contribution statement

Wenjie Liu: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Data curation. **Zhijie Ren:** Writing – review & editing, Visualization, Software, Methodology, Conceptualization. **Liang Chen:** Writing – review & editing, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (62071240), the Innovation Program for Quantum Science and Technology (2021ZD0302901), the Natural Science Foundation of Jiangsu Province (BK20231142), and the Priority Academic Program Development of Jiangsu Higher Education Institutions (PAPD).

Data availability

Data will be made available on request.

References

- [1] S. Ji, S. Pan, E. Cambria, A survey on knowledge graphs: Representation, acquisition, and applications, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (2) (2022) 494–514.
- [2] Y. Cao, X. Wang, X. He, Z. Hu, T.-S. Chua, Unifying knowledge graph learning and recommendation: Towards a better understanding of user preferences, in: *The World Wide Web Conference, WWW '19*, 2019, pp. 151–161.
- [3] L.Y. Zeng X, Toward better drug discovery with knowledge graph, *Curr. Opin. Struct. Biol.* 72 (2022) 114–126.
- [4] A. Bordes, N. Usunier, A. Garcia-Durán, J. Weston, O. Yakhnenko, Translating embeddings for modeling multi-relational data, in: *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2*, NIPS '13, 2013, pp. 2787–2795.
- [5] Y. Lin, Z. Liu, M. Sun, Learning entity and relation embeddings for knowledge graph completion, in: *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence*, AAAI '15, 2015, pp. 2181–2187.
- [6] T. Ebisu, R. Ichise, Toruse: Knowledge graph embedding on a Lie group, in: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*, AAAI '18, 2018, pp. 1819–1826.
- [7] Z. Sun, Z. Deng, J. Nie, Rotate: Knowledge graph embedding by relational rotation in complex space, in: *Proceedings of the 7th International Conference on Learning Representations*, 2019.
- [8] M. Nickel, V. Tresp, H. Kriegel, Factorizing YAGO: scalable machine learning for linked data, in: *Proceedings of the 21st International Conference on World Wide Web*, WWW '12, 2012, pp. 271–280.
- [9] B. Yang, W. Yih, X. He, Embedding entities and relations for learning and inference in knowledge bases, in: *Proceedings of the 3rd International Conference on Learning Representations, ICLR 2015*, 2015.
- [10] T. Trouillon, J. Welbl, S. Riedel, Complex embeddings for simple link prediction, in: *Proceedings of the 33rd International Conference on Machine Learning, ICML '16*, 2016, pp. 2071–2080.
- [11] I. Balazevic, C. Allen, T.M. Hospedales, Tucker: Tensor factorization for knowledge graph completion, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing and 9th International Joint Conference on Natural Language Processing*, 2019, pp. 5184–5193.
- [12] T. Dettmers, P. Minervini, P. Stenetorp, Convolutional 2D knowledge graph embeddings, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, AAAI '18, 2018, pp. 1811–1818.
- [13] D.Q. Nguyen, T.D. Nguyen, D.Q. Nguyen, A novel embedding model for knowledge base completion based on convolutional neural network, in: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT*, 2018, pp. 327–333.
- [14] X. Jiang, Q. Wang, B. Wang, Adaptive convolution for multi-relational learning, in: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2019, pp. 978–987.
- [15] S. Ji, S. Pan, E. Cambria, Marttinen, A survey on knowledge graphs: Representation, acquisition, and applications, *IEEE Trans. Neural Netw. Learn. Syst.* 33 (2) (2022) 494–514.
- [16] Y. Lin, Z. Liu, H. Luan, M. Sun, S. Rao, S. Liu, Modeling relation paths for representation learning of knowledge bases, in: *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, 2015, pp. 705–714.
- [17] R. Das, A. Neelakantan, D. Belanger, A. McCallum, Chains of reasoning over entities, relations, and text using recurrent neural networks, in: *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 2017, pp. 132–141.
- [18] W. Xiong, T. Hoang, W.Y. Wang, DeepPath: A reinforcement learning method for knowledge graph reasoning, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2017, pp. 564–573.
- [19] Y. Wang, H. Wang, J. He, W. Lu, S. Gao, Tagat: Type-aware graph attention networks for reasoning over knowledge graphs, *Knowl.-Based Syst.* 233 (2021) 107500.
- [20] Z. Wu, S. Pan, F. Chen, A comprehensive survey on graph neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (1) (2021) 4–24.
- [21] W.L. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Proceedings of the Conference on Neural Information Processing Systems, NIPS '17*, 2017, pp. 1024–1034.
- [22] M.S. Schlichtkrull, T.N. Kipf, P. Bloem, Modeling relational data with graph convolutional networks, in: *Proceedings of the Extended Semantic Web Conference*, in: *Lecture Notes in Computer Science*, vol. 10843, 2018, pp. 593–607.

- [23] S. Vashishth, S. Sanyal, V. Nitin, Composition-based multi-relational graph convolutional networks, in: *Proceedings of the International Conference on Learning Representations, ICLR '20*, 2020.
- [24] D. Nathani, J. Chauhan, C. Sharma, Learning attention-based embeddings for relation prediction in knowledge graphs, in: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 2019, pp. 4710–4723.
- [25] X. Wang, H. Ji, C. Shi, Heterogeneous graph attention network, in: *Proceedings of the International World Wide Web Conference, WWW '19*, 2019, pp. 2022–2032.
- [26] S. Zheng, S. Mai, Y. Sun, H. Hu, Y. Yang, Subgraph-aware few-shot inductive link prediction via meta-learning, *IEEE Trans. Knowl. Data Eng.* 35 (6) (2023) 6512–6517.
- [27] S. Vashishth, S. Sanyal, V. Nitin, Interact: Improving convolution-based knowledge graph embeddings by increasing feature interactions, in: *Proceedings of the Association for the Advance of Artificial Intelligence*, vol. 34, 2020, pp. 3009–3016.
- [28] R. Das, S. Dhuliawala, M. Zaheer, L. Vilnis, I. Durugkar, A. Krishnamurthy, A. Smola, A. McCallum, Go for a walk and arrive at the answer: Reasoning over paths in knowledge bases using reinforcement learning, in: *Proceedings of the 6th International Conference on Learning Representations, ICLR '18*, 2018.
- [29] Y. Geng, J. Chen, J.Z. Pan, M. Chen, S. Jiang, W. Zhang, H. Chen, Relational message passing for fully inductive knowledge graph completion, in: *Proceedings of the 39th IEEE International Conference on Data Engineering, KDD '21*, IEEE, 2023, pp. 1221–1233.
- [30] A. Sadeghian, M. Armandpour, P. Ding, D.Z. Wang, DRUM: End-to-end differentiable rule mining on knowledge graphs, in: *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, 2019, pp. 15321–15331.
- [31] R. Ye, X. Li, Y. Fang, A vectorized relational graph convolutional network for multi-relational network alignment, in: *Proceedings of the 28th International Joint Conference on Artificial Intelligence, IJCAI '19*, 2019, pp. 4135–4141.
- [32] P. Velickovic, G. Cucurull, A. Casanova, Graph attention networks, in: *Proceedings of the International Conference on Learning Representations*, 2018.
- [33] D. Nathani, J. Chauhan, C. Sharma, Learning attention-based embeddings for relation prediction in knowledge graphs, in: *Proceedings of the Association for Computational Linguistics, Florence, Italy*, 2019, pp. 4710–4723.
- [34] Z. Zhu, Z. Zhang, L.A.C. Xhonneux, Neural bellman-ford networks: A general graph neural network framework for link prediction, in: *Proceedings of the 35th International Conference on Neural Information Processing Systems*, 2021, pp. 29476–29490.
- [35] Y. Zhang, Q. Yao, Knowledge graph reasoning with relational digraph, in: *Proceedings of the ACM Web Conference 2022, WWW '22*, 2022, pp. 912–924.
- [36] Y. Wu, J. Zhou, A neighborhood-aware graph self-attention mechanism-based pre-training model for knowledge graph reasoning, *Inform. Sci.* 647 (2023) 119473.
- [37] K. Toutanova, D. Chen, P. Pantel, Representing text for joint embedding of text and knowledge bases, in: *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2015, pp. 1499–1509.
- [38] G. Balloccu, L. Boratto, G. Fenu, M. Marras, Reinforcement recommendation reasoning through knowledge graphs for explanation path quality, *Knowl.-Based Syst.* 260 (2023) 110098.